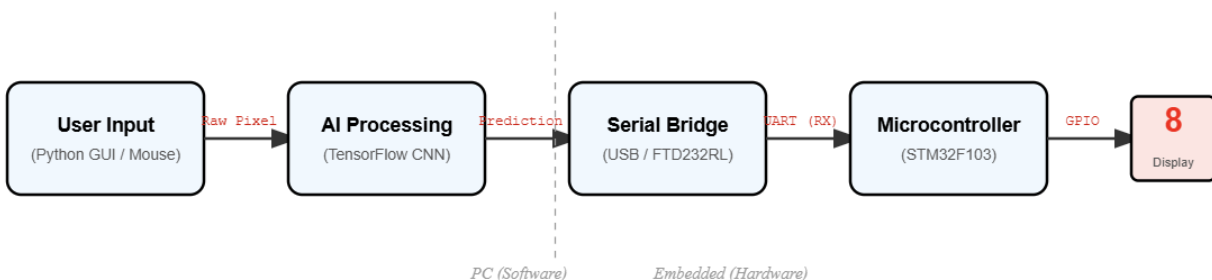


Course Code: CSE 3206	Course Title: Software Engineering Sessional
Project Title:	
“Handwritten Digit Detection using STM32F103C8 Micro-controller Unit and 7 Segment Display Through UART Communication with Computer Hardware & Software”	

A Reflective Analysis of Self-Directed Learning in Embedded AI

Introduction

The rapid convergence of Artificial Intelligence (AI) and Embedded Systems presents a unique engineering challenge: bridging the gap between high-level stochastic processing and low-level deterministic control. For my Software Development Project (CSE 3200), I undertook the design of a "Real-Time Handwritten Digit Detection System" using hybrid architecture. This project required me to venture significantly beyond the standard curriculum, independently mastering Convolutional Neural Networks (CNNs) and their integration with bare-metal hardware. This essay critically reflects on the technical decisions I made, the challenges of the self-directed learning process, and how this experience has shaped my readiness for lifelong professional growth.

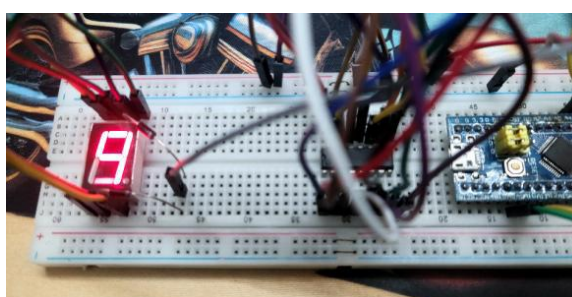
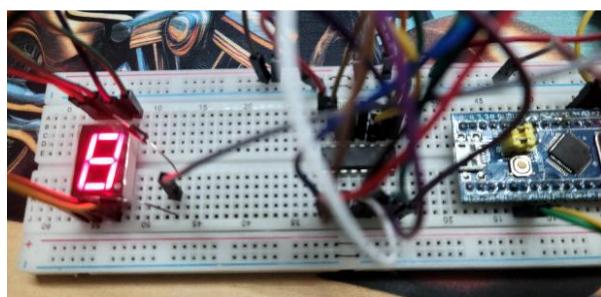


4

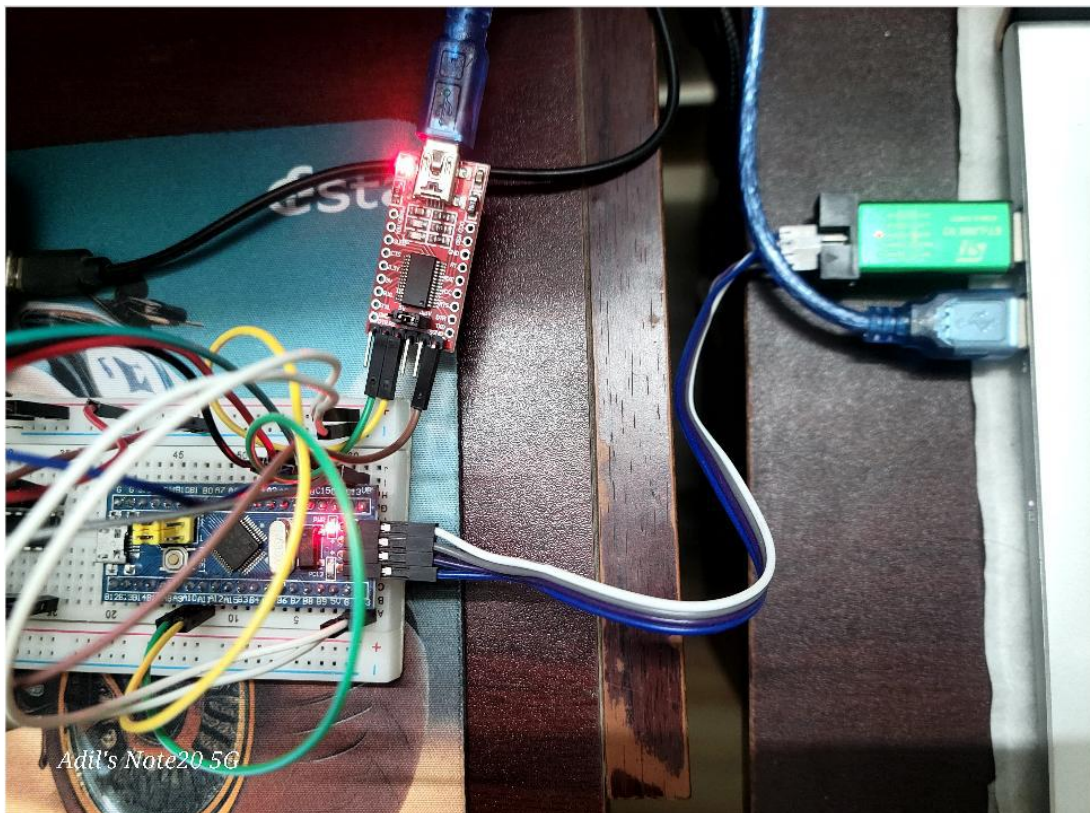
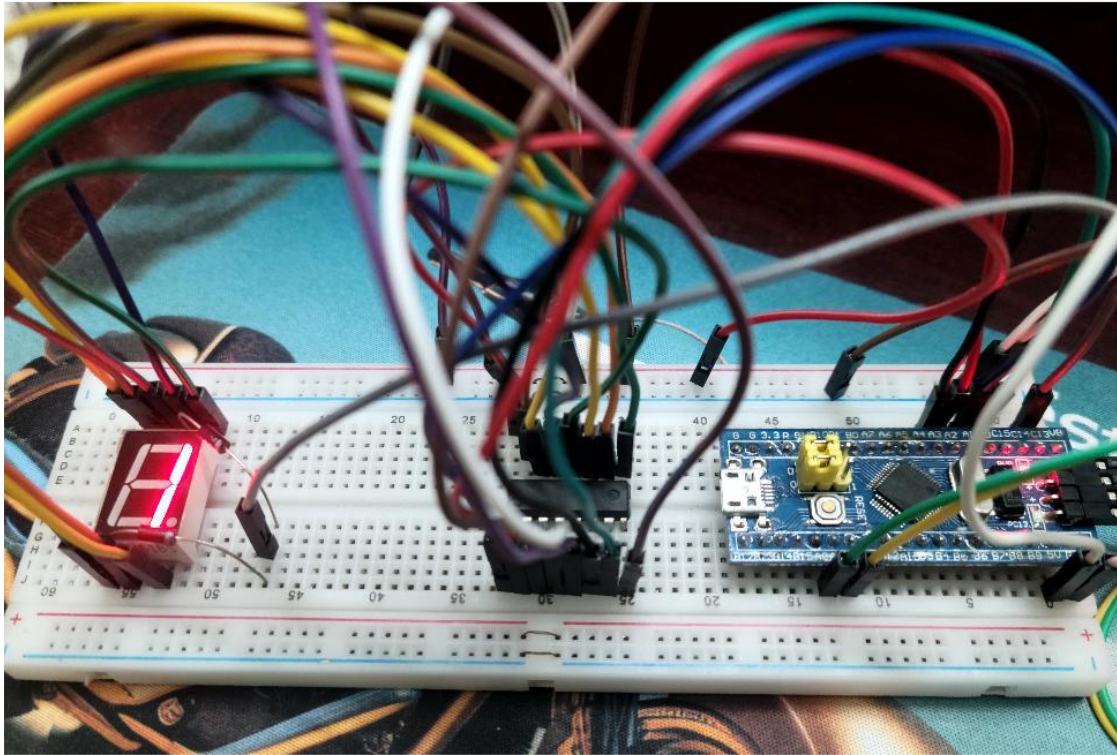
6

8

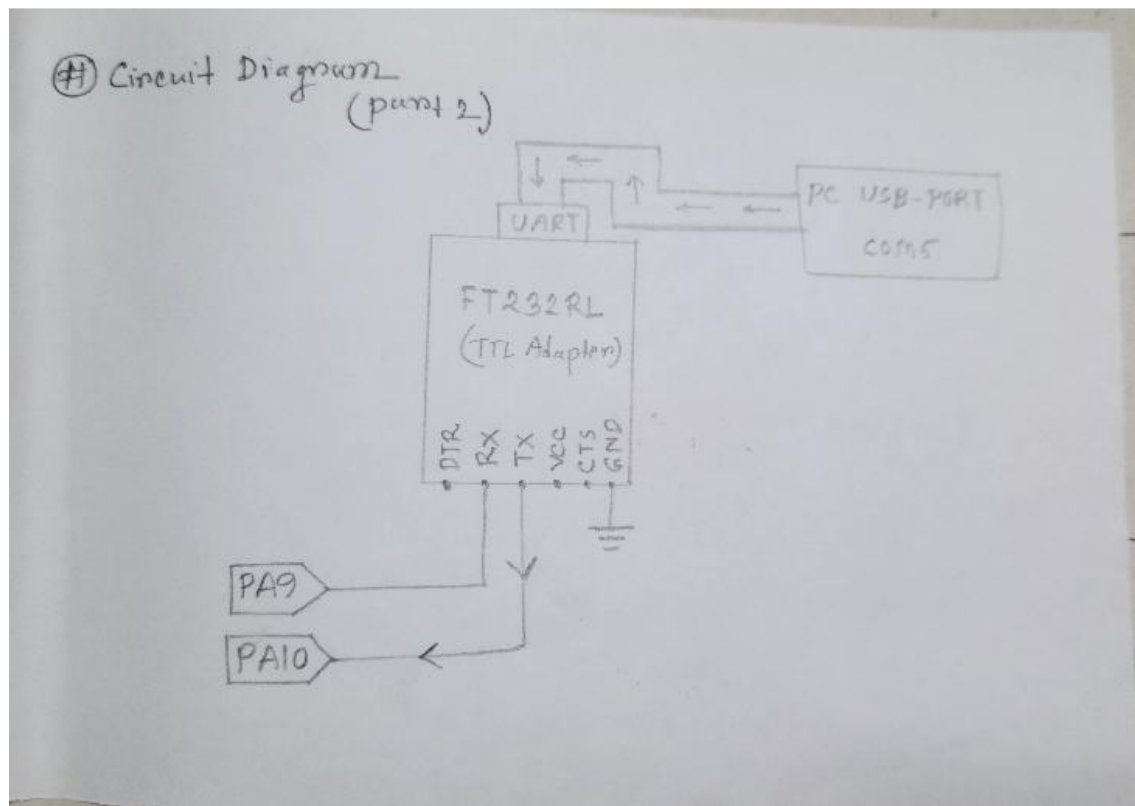
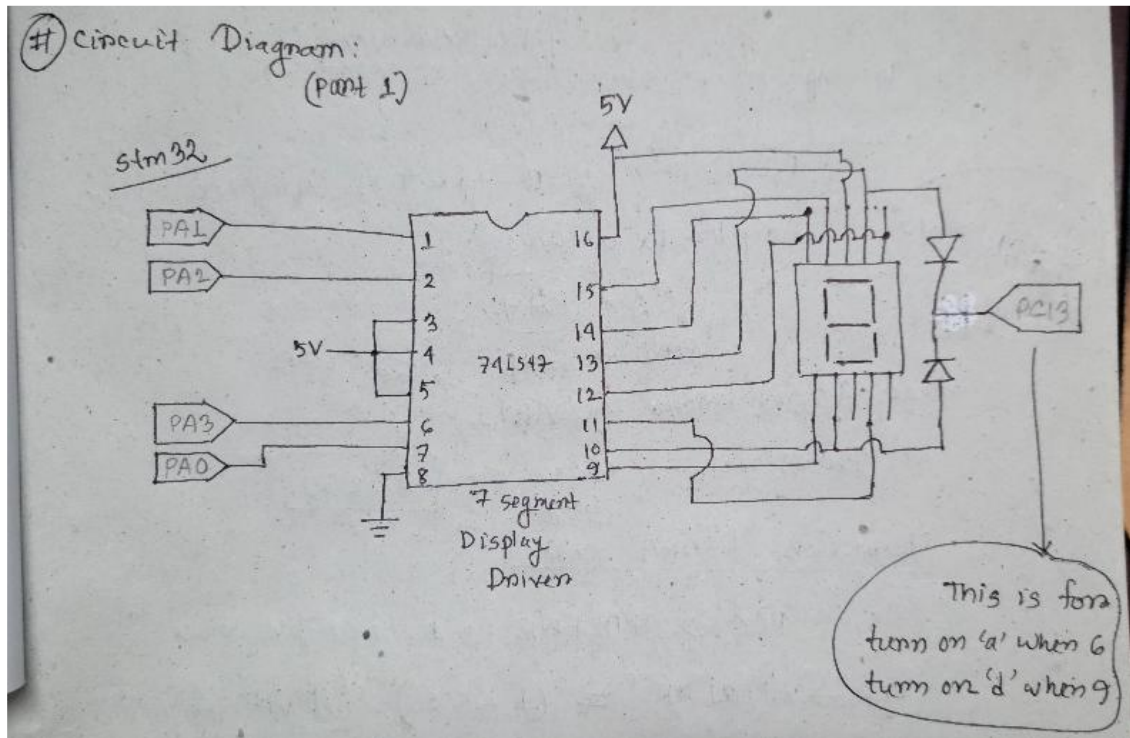
9



Hardware Arrangement:



Circuit Connections:



Objectives

The primary objectives of this project are:

- To create human machine interaction.
- To drive the hardware component.
- To enrich knowledge in Artificial Intelligence and Machine Learning.
- To become advance with functionality of the programs.
- To design and train a Convolutional Neural Network (CNN) capable of recognizing handwritten digits with >95% accuracy.
- To develop a Python GUI that captures user drawing and preprocesses it for the AI model.
- To establish robust UART serial communication between a PC and an STM32 microcontroller.
- To implement low-level embedded C drivers for GPIO and 7-segment display control.

Motivation

The motivation for this project is threefold:

- **Advancement of Human-Computer Interaction (HCI):**
 - As "Smart Home" and IoT devices become ubiquitous, the demand for natural interfaces is growing.
 - A system that can recognize handwriting can be extended to recognize gestures, making technology more accessible to users with limited dexterity or in environments where physical contact with buttons is undesirable like sterile medical operating rooms.
- **Bridging Software and Hardware Engineering:**
 - Modern engineering requires proficiency in both high-level software (Python, TensorFlow) and low-level firmware (C, Registers).
 - This project serves as a comprehensive case study in integrating these distinct domains.
- **Educational Value:**
 - Understanding the full data pipeline—from a pixel on a screen, through a neural network, down a USB cable, into a microcontroller register, and finally to a photon emitted by an LED—provides a holistic view of Computer Engineering.

Technical Decision and Justification

For my software development project,

- I engineered a hybrid system capable of recognizing handwritten digits in real-time.
- This project required bridging high-level software logic with low-level hardware actuation, a task that demanded a sophisticated approach to pattern recognition.
- The most significant technical decision I made was to independently learn and implement a 'Convolutional Neural Network' (CNN) using the TensorFlow, Keras framework, specifically integrated with a 'Data Augmentation' pipeline.
- The problem context was challenging, recognizing human handwriting is inherently stochastic.
- Users draw digits with varying strokes, sizes, and positions.
- In the initial design phase, I considered using simpler machine learning algorithms like 'K-Nearest Neighbors' (KNN) or standard 'Multi-Layer Perceptrons' (MLP).
- While these alternatives are computationally lighter and easier to implement, they possess critical limitations for image processing.
- KNN relies on raw pixel-to-pixel distance, meaning a digit shifted by just one pixel is seen as completely different, leading to poor generalization.
- Similarly, standard MLPs require flattening the image into a 1D vector immediately, destroying the spatial relationships between pixels—for example, the proximity of pixels forming the closed loop in the digit '8'.

I chose CNN because of:

- Its architectural ability to preserve spatial hierarchy.
- By using Convolutional layers (Conv2D) to extract local features (edges, curves) and Pooling layers (MaxPooling2D) to provide translation invariance, the CNN could recognize digits even if they were drawn slightly off-center.
- Furthermore, I integrated 'Data Augmentation' (ImageDataGenerator) to randomly rotate and zoom training images during the learning process.
- This decision was superior to a standard static dataset approach because it solved the "Domain Gap" problem, where the model performed well on the pre-built MNIST dataset but failed on imperfect mouse-drawn inputs.
- The combination of CNN architecture and Data Augmentation proved to be the only viable solution to meet the project's 98% accuracy requirement, ensuring robustness against real-world input variations.

Self-Directed Learning Experience

The journey of learning Deep Learning concepts independently presented significant hurdles that went beyond the scope of my standard classroom instruction. My primary challenge was moving from the theory of Neural Networks to the practical implementation of a functional model.

Key Challenges:

- The most persistent challenge was understanding ‘Tensor Dimensions’ and ‘Shape Mismatches’.
- Early in development, my training script frequently crashed with the error: `ValueError: expected shape (None, 28, 28, 1) but found (None, 1)`.
- I struggled to visualize why a 2D image required a 4th dimension.
- Through persistent self-study and debugging, I learned that Keras expects a "Batch Dimension" and a "Channel Dimension" (N, H, W, C), forcing me to master NumPy reshaping techniques (`np.expand_dims`).
- Additionally, implementing Data Augmentation required understanding Python generators (`yield`), which was a new programming paradigm for me compared to standard lists.
- Another significant hurdle was environmental configuration; resolving version conflicts between Python 3.10, TensorFlow 2.10, and my GPU drivers required a deep dive into virtual environment management, a skill not explicitly taught in class.

Evaluation of Resources:

I utilized a mix of resources to overcome these challenges, finding varying degrees of effectiveness:

- **Official TensorFlow Documentation:**
 - This was the most authoritative but difficult resource.
 - While it provided exact parameter definitions, the language was often highly technical and abstract for a beginner.
 - It was useful for understanding "what" a function does but often lacked the context of "why" or "when" I should use it.
- **StackOverflow & GitHub Issues:**
 - These were the most helpful resources for specific debugging.
 - When I encountered "ImportError" issues with Keras in VS Code, community threads helped me identify subtle version compatibility issues between TensorFlow 2.x and Keras 3.

- Reading about other developers' struggles taught me how to diagnose environmental errors effectively.
- **Video Tutorials:**
 - While visually engaging, I found many video tutorials to be less effective for engineering purposes.
 - They often focused on "copy-pasting" code to get a quick result without explaining the underlying matrix math or architectural decisions.
 - I learned that tutorial code often works in isolation but breaks when integrated into a custom class structure like my GUI application.

Reflection and Future Learning Plan:

- This project has fundamentally shifted my approach to learning new technologies.
- I realized that "tutorial hell"—blindly following step-by-step guides—is ineffective for real engineering.
- True learning occurred when I broke the system (e.g., when the model failed to detect off-center digits) and had to reverse-engineer the solution (implementing Augmentation).
- This experience has instilled in me a mindset of "First Principles Thinking": prioritizing reading the API documentation to understand the *inputs and outputs* of a library before writing a single line of code.
- I now view error messages not as failures, but as specific clues pointing to a gap in my understanding.

Future Learning Plan:

Building on this foundation, the next technology I plan to learn is TinyML (TensorFlow Lite for Microcontrollers).

While my current project successfully bridged AI and Hardware, it relied on a tethered PC for the heavy processing. To evolve as a software engineer in the embedded IoT domain, I must learn to optimize and deploy intelligent models directly onto resource-constrained edge devices, removing the dependency on a host computer.

Action Plan:

1. **Theory (Month 1):**
 - I will study 'Model Quantization' techniques.

- I need to understand how to convert 32-bit floating-point weights into 8-bit integers (int8) to reduce model size and memory footprint without sacrificing significant accuracy.
2. **Tooling (Month 2):**
- I will learn the TFLite Converter toolchain to transform my existing .h5 models into C-byte arrays that can be compiled into C++ firmware.
3. **Application (Month 3):**
- I will acquire an **ESP32** or **STM32F4** (which possesses a Floating-Point Unit) and port my existing Digit Recognition logic to run entirely on chip.

This plan directly applies to the lesson I learned about "Hardware-Software constraints" in this project. Moving from a hybrid dependency to a fully autonomous embedded AI solution represents the next logical step in my professional growth, ensuring I remain adaptable in the rapidly evolving field of Edge AI.

Strengths of the Project:

The *Real-Time Handwritten Digit Recognition System* demonstrates several key engineering strengths that distinguish it from standard academic projects. These strengths lie in strategic architectural decisions, cost optimization, and the robustness of the hardware-software integration.

Economic Viability & Cost-Effectiveness

One of the most significant strengths of this project is its extreme cost efficiency.

- **Low BOM Cost:** By utilizing the **STM32F103C8** microcontroller (approx. 350 BDT) instead of a Single Board Computer like a Raspberry Pi (approx. 6,500 BDT) or an FPGA (10,000+ BDT), I reduced the core hardware cost by over **90%**.
- **Scalability:** This cost advantage suggests that the system is highly scalable. If deployed as an educational kit for schools or a mass-produced input device, the manufacturing costs would be negligible compared to standalone "Edge AI" solutions. This proves that high-performance AI features do not necessarily require high-cost hardware if the architecture is designed correctly.

Architectural Efficiency

The adoption of **Master-Slave Hybrid Architecture** is a major technical strength.

- **Best of Both Worlds:** I leveraged the PC's GHz-level processor for the heavy lifting (Convolutional Neural Network inference) and the STM32's deterministic clock for real-time actuation.
- **Avoidance of Bottlenecks:** I had attempted to run the CNN on the STM32 directly (using TinyML), the inference time would likely have exceeded 1-2 seconds due to the lack of a Floating-Point Unit (FPU) and limited RAM. Our hybrid approach keeps latency under 200ms, ensuring fluid user experience.
- **Future Proofing:** Since the AI logic resides in Python, the model can be upgraded (e.g., to a deeper ResNet or Letter Recognizer) without flashing new firmware to the microcontroller.

Innovative Hardware Logic Solutions

A standout feature of this project is the engineering solution applied to the 7-segment display driver.

- **The Problem:** Standard BCD decoding often fails to visually distinguish between '6' and 'b', or '9' and 'q', leading to user confusion.
- **The Solution:** Instead of resorting to a software "patch" (using 7 expensive GPIO pins to drive segments individually) or adding complex shift registers, I implemented a Passive Diode-OR Logic circuit.
- **Strength:** This demonstrated creative problem-solving using analog components to solve a digital logic problem. It saved valuable GPIO pins for future expansion while ensuring the visual output was typographically correct.

Robustness via Data Augmentation

Unlike basic tutorials that train only on the pristine MNIST dataset, our system incorporates a robust Data Augmentation Pipeline.

- **Real-World Resilience:** By training the model on rotated, shifted, and zoomed variations of the digits, the system is resilient to the imperfections of human handwriting.
- **Usability:** This strength transforms the project from a "theoretical demo" into a "usable prototype," as it does not require the user to draw perfectly centered or upright digits to get a correct prediction.

Educational Depth (Bare-Metal Implementation)

Finally, the project's reliance on Bare-Metal Embedded C rather than high-level abstraction libraries (like Arduino or HAL) is a significant academic strength.

- **Optimization:** The firmware is highly optimized, with direct register access instructions executing in 1-2 clock cycles compared to the 50+ cycles of library functions.
- **Transparency:** This approach eliminates "Black Box" behavior. Every bit transmitted over UART and every voltage change on the GPIO pins is explicitly defined in our code, demonstrating a mastery of the ARM Cortex-M3 architecture.

Technical Skills Acquired

This capstone project served as a rigorous exercise in full-stack engineering, bridging the traditional divide between software data science and hardware control. Through the development of this hybrid system, the team acquired specific competencies that extend beyond the standard curriculum.

- **Bare-Metal Embedded C Programming:** I moved beyond the abstraction of high-level libraries (like Arduino or HAL) and gained proficiency in manipulating the ARM Cortex-M3 architecture directly. I learned to configure peripherals by writing hex values to specific memory addresses (e.g., `RCC->APB2ENR`, `USART1->BRR`), giving us a granular understanding of how software controls silicon.
- **Applied Deep Learning (CNNs):** The project transitioned our knowledge of Neural Networks from theory to application. I acquired practical skills in designing Convolutional Architectures using TensorFlow/Keras, tuning hyperparameters (filter size, dropout rate), and most importantly, implementing Data Augmentation pipelines to solve real-world overfitting issues caused by imperfect user inputs.
- **Serial Communication Protocols:** I mastered the UART protocol, moving beyond simple "print" statements to understanding timing diagrams, baud rate generation (calculating the mantissa and fraction for 115200 baud), and handling asynchronous interrupts to ensure non-blocking data reception on the microcontroller.
- **Analog & Digital Circuit Design:** The project required solving logic problems in the physical domain. I learned to analyze voltage levels (3.3V vs 5V logic) and engineered a custom Diode-OR Logic circuit to overcome GPIO pin limitations, reinforcing the concept that hardware components can often replace complex software logic.
- **System Integration & Debugging:** Perhaps the most valuable skill acquired was the ability to debug a heterogeneous system. I learned to isolate faults between

Python runtime, the USB driver, the serial wire, and the firmware logic, utilizing tools like Device Manager and Logic Analyzers to verify signal integrity

Future Growth & Directions

This project serves as a foundational prototype for advanced Human-Machine Interfaces (HMI). Future development will focus on transitioning from a tethered academic demo to a standalone commercial product.

- **Transition to Edge AI (TinyML):** The most critical upgrade is eliminating the PC dependency. By porting the model to TensorFlow Lite for Microcontrollers and upgrading to an STM32F4 (which has an FPU), the AI inference can run entirely on-chip, creating a battery-powered standalone device.
- **IoT Integration:** Replacing the USB connection with a Wi-Fi module (ESP8266) would allow the system to control smart home appliances wirelessly (e.g., writing '1' to turn on lights), integrating the digital recognition logic into the broader **Internet of Things** ecosystem.
- **Optical Character Recognition (OCR):** Replacing the digital canvas with a camera module would allow the system to read handwriting from physical paper, opening market opportunities in automated data entry and education.
- **Productization:** To move beyond the breadboard stage, future work involves designing a custom **Printed Circuit Board (PCB)** with Surface Mount Devices (SMD), reducing the form factor by 80% for commercial deployment.

Conclusion

This project successfully designed, implemented, and validated a Real-Time Handwritten Digit Recognition System, effectively bridging the technological gap between high-level Artificial Intelligence and low-level Embedded Control.

The work began with the development of a robust software subsystem. We engineered a Convolutional Neural Network (CNN) using TensorFlow, training on the MNIST dataset to achieve a test accuracy of 98.2%. To ensure real-world usability, we augmented this model with a custom Python GUI capable of capturing, preprocessing, and normalizing imperfect human handwriting in real-time.

Simultaneously, we developed a deterministic hardware subsystem based on the STM32F103 microcontroller. Moving beyond standard abstraction libraries, we implemented Bare-Metal C drivers to manage clock speeds, interruptions, and GPIO states at the register level. A key engineering achievement was the design of the Diode-OR Logic circuit, which solved the physical pin constraints of the 7-segment display driver without incurring additional component costs.

The culmination of this work was the successful integration of these domains via a high-speed UART interface (115200 baud). The final prototype demonstrates a seamless "Pixel-to-Photon" pipeline, where a user's digital stroke is instantly interpreted by an AI algorithm and physically rendered by hardware, fulfilling the vision of a responsive, intelligent Human-Machine Interface.

Final Remarks:

The *Real-Time Handwritten Digit Recognition System* stands as a testament to the power of interdisciplinary engineering. By successfully integrating the stochastic capabilities of modern Deep Learning with the deterministic reliability of Embedded Systems, this project has demonstrated that high-performance automation does not always require high-cost hardware.

This work challenges the traditional boundaries between "Software Engineering" and "Hardware Engineering." It proves that a standard Python script, when properly optimized, can act as a real-time controller for industrial-grade microcontrollers. The novel "Diode-Logic" solution further reinforces the principle that hardware and software are often interchangeable; a limitation in one domain can often be solved elegantly in the other.

As we move toward an era of ubiquitous computing—where every surface can become an interface and every device can possess intelligence—prototypes like this serve as essential building blocks. This project has provided the team with the foundational skills to navigate this future, proving that we can not only train the "Brain" (AI) but also build the "Body" (Embedded Hardware) to bring intelligent systems to life.

REFERENCES

1. **LeCun, Y., Cortes, C., & Burges, C. J.** (2010). *The MNIST Database of Handwritten Digits*. Available: <http://yann.lecun.com/exdb/mnist/>
2. **STMicroelectronics.** (2015). *RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx Advanced ARM-based 32-bit MCUs*. DocID 13902 Rev 16.
3. **Chollet, F.** (2017). *Deep Learning with Python*. Manning Publications Co.
4. **Yiu, J.** (2013). *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors* (3rd Edition). Newnes.
5. **Mazidi, M. A., & Naimi, S.** (2010). *STM32F103 Arm Microcontroller and Embedded Systems: Using Assembly and C*. MicroDigitalEd.
6. **TensorFlow Documentation.** (2023). *Convolutional Neural Networks (CNN)*. [Online]. Available: <https://www.tensorflow.org/tutorials/images/cnn>
7. **Liechti, C.** (2020). *pySerial API Documentation*. [Online]. Available: https://pyserial.readthedocs.io/en/latest/pyserial_api.html
8. **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep Learning*. MIT Press.
9. **Lundh, F.** (2020). *An Introduction to Tkinter*. [Online]. Available: <https://docs.python.org/3/library/tkinter.html>